

作業ログ

報告者：佐藤涼菜

報告日：6 月 23 日

プロジェクト名：マジカル☆観覧車

1. 進捗サマリー（要約）

- 全体進捗率：85[%]
- マイルストーン達成状況：おおよそ達成（テストフェーズは結合待ち）
- 主要成果：
 - 指揮デバイスの最終固定および加工をし，本体・センサの実装（No.2112）を完了
 - BPM・音量変化機構の実装（No.2113）を完了
 - 計画書との変更点の記述（No.2231）
- 重大課題（影響度：高）：
 - テストフェーズ（No.3241）が結合待ちのため未着手

2. 作業ログ（詳細トラッキング）

2.1 作業ログ

表 1: 作業ログ

No.	作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロッカー	コメント
2112	本体・センサの実装	5/24	6/17	完了	100%	無	回路の最終固定と筐体の加工を行った。
2113	BPM・音量変化機構の実装	5/29	6/1	完了	100%	無	スライド式可変抵抗器の動作確認を実施し完了。
3241	LED マトリックスの反映確認	6/17	6/30	未着手	0%	有	結合テストの実施待ち。6/24 の授業内にて実施予定。
-	作業ログの記入	6/17	6/23	完了	100%	無	特になし。

2.2 日ごとの作業ログ

表 2: 日ごとの作業ログ

日付	No.	作業項目	時間	開始時の状態	終了時の状態	進捗率	コメント
6/17	2113	BPM・音量変化機構の実装	1.5h	おおよそ完了	完了	100%	スライド式可変抵抗器の再はんだ付けおよび動作確認を実施し完了.
6/18	2112	本体・センサの実装	2h	進行中	進行中	93%	筐体の改修に着手. スライド式可変抵抗器とタクトスイッチのみを露出させる加工を開始.
6/19	2112	本体・センサの実装	2h	進行中	進行中	96%	操作部の露出加工を継続. 内部配線の整理を実施.
6/20	2112	本体・センサの実装	1.5h	進行中	進行中	98%	回路の最終固定と筐体内部の配線固定を実施.
6/21	2112	本体・センサの実装	1.5h	進行中	完了	100%	筐体の最終仕上げを行い, 本体・センサの実装を完了.
6/22	2231	LED マトリクス制御	1h	完了	完了	100%	計画書 (Version 1.3) と実装の変更点を整理し記述した.
6/23	-	作業ログの記入	1h	未着手	完了	100%	特になし.

3. 計画時の GC と現在の GC

計画段階のガントチャート (GC) を図 1 (実装フェーズ) および図 2 (テストフェーズ) に示す.

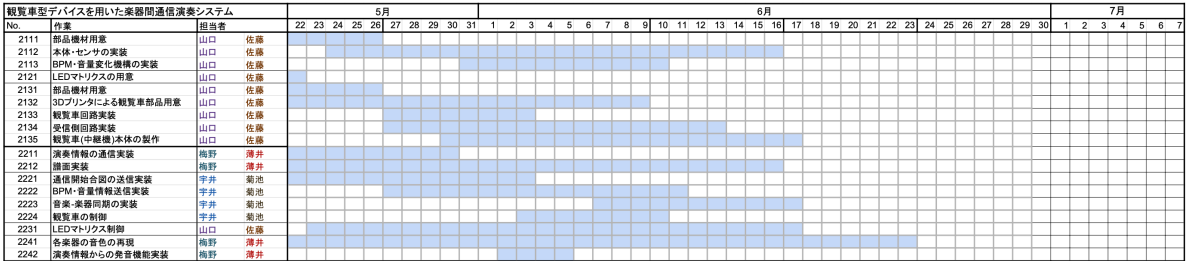


図 1 計画段階のガントチャート (実装フェーズ)

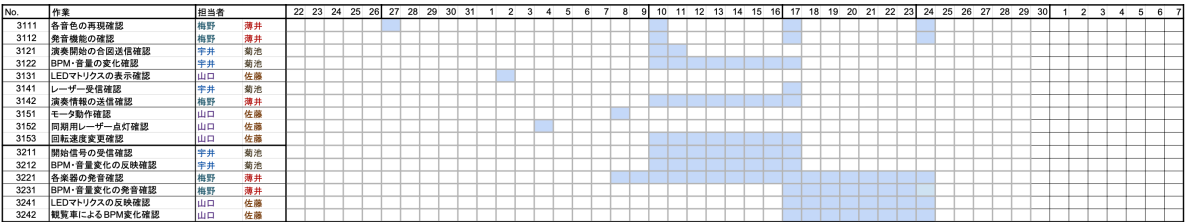


図 2 計画段階のガントチャート (テストフェーズ)

次に、現在の GC を図 3 に示す。達成した項目は緑色で塗られている。

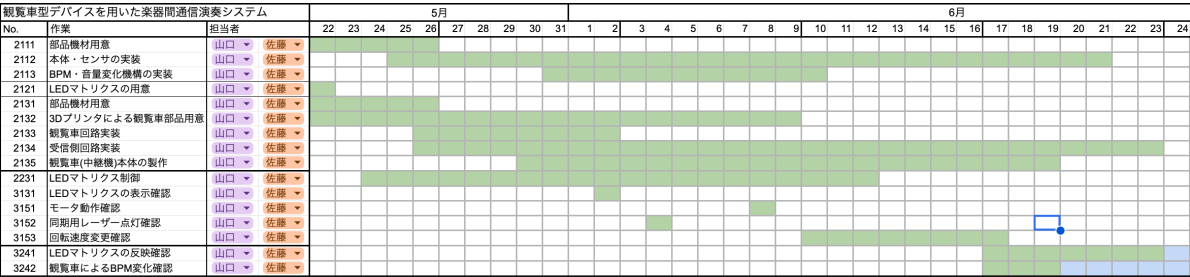


図 3 現在のガントチャート

4. 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
可変抵抗器がブレッドボード上で安定接続できない(No.2112/2113に影響)	スライド式可変抵抗器の端子形状がブレッドボードに適合しないため、	高	高	スライド式可変抵抗器を基盤に設置し、筐体へ固定した、	6/12	対応済
1 機のスライド式可変抵抗器のはんだ付け設置不良 (No.2112 に影響)	基盤へのはんだ付け時に接触不良が発生したため、	中	高	6/17 に再はんだ付けを実施し、動作を確認した、	6/17	対応済
テストフェーズが結合待ち (No.3241 に影響)	通信系との結合が完了していないため、LED マトリクスの反映確認テストが実施できない、	高	中	通信系の準備完了を待ち、結合テストを実施する、	6/30	対応中

※影響度＝プロジェクトへのインパクト

※優先度＝対応の緊急性

5. AI 利用状況と検証（再現性重視）

今回は AI を利用していない、

6. No.2231 LED マトリクス制御：計画書との変更点

計画書（2026 年 06 月 12 日版 Version 1.3）のセクション 3.5.8 および変更履歴（2026 年 06 月 19 日版 Version 1.4）を参照し、Display（LED マトリクス制御）の実装における計画書からの変更点を以下に示す、

6.1 描画 API の変更

計画書（3.5.8.3）では、フレームバッファの描画に `matrix.renderBitmap()` を使用すると記述されている、`renderBitmap()` は `uint8_t[8][12]` 形式の配列を引数に取る関数である、実装では `uint32_t[3]` 形式にビットパックし、`matrix.loadFrame()` で描画する方式に変更した、これは、`loadFrame()` の方がフレームバッファのメモリ効率が高く、96 ビット（8 行×12 列）を 3 つの 32 ビット整数で表現できるためである、

6.2 関数の追加

計画書の表 3.33 には `checkUDP` と `displayBPM` の 2 関数のみ記載されている。実装では以下の 3 関数を追加した。

- `clearDisplay()`: 演奏終了ヘッダ ('E') 受信時に LED マトリクスを全消灯する関数。計画書の通信プロトコル (表 3.29, 3.30) にはヘッダ 'E' が定義されているが、Display 側の関数一覧にはこの処理への言及がなかったため追加した。
- `drawDigit()`: 1 桁の数字を独自フォント (`font3x5[]`) を参照してフレームバッファに描画する内部関数。`displayBPM()` から桁ごとに呼び出される。
- `displaySetup()`: `matrix.begin()` を Display.cpp 側で呼ぶための初期化関数。`Arduino_LED_Matrix` の表示バッファはヘッダ内で static 定義されており、include する翻訳単位ごとに別実体となる。そのため、描画を行う `loadFrame()` と同一ファイルから `begin()` を呼び出す必要があり、本関数を追加した。

6.3 モジュール構成の変更

計画書の図 3.16 では `checkUDP()` は Display.ino 側に配置されている。実装では Display.cpp に移動した。`checkUDP()` 内で `currentBPM` (Display.cpp の静的変数) を参照する必要があるためである。これにより、Display.ino には `setup()` と `loop()` のみが残る構成となった。

6.4 受信データの読み込み形式

計画書 (3.5.8.3) では「ペイロードを文字列として読み込む」と記述されている。実装では `uint8_t buffer[2]` としてバイナリ形式で読み取り、`buffer[0]` をヘッダ文字 (`char`)、`buffer[1]` を段階番号 (`uint8_t`) として処理している。通信プロトコル仕様 (表 3.29, 3.30) 自体は固定長 2 バイトのバイナリ形式で定義されており、実装側がプロトコル仕様に忠実な形式を採用した。

6.5 静的 IP 設定の追加

計画書の表 3.24 には Display 側の定数として `localIP` (観覧車デバイス自身の静的 IP アドレス) が定義されている。実装では `WiFi.config(localIP, gateway, subnet)` を `WiFi.begin()` の前に呼び出すことで、指揮デバイスの送信先 IP アドレス (192.168.11.10) と一致する固定 IP を設定した。

7. 次回計画

- 目標：通信系との結合テストの実施
 - 通信系の準備が完了次第、LED マトリクスの反映確認テスト (No.3241) を実施する。
 - 指揮デバイスからの UDP 送信により BPM 値が LED マトリクスに正しく表示されることを確認する。
- 達成基準：
 - 指揮デバイスから BPM 値を送信し、表示デバイスの LED マトリクスに正しく反映されること。
 - 演奏終了ヘッダ ('E') の受信時に LED マトリクスが消灯すること。
- 期限：
 - 2026/6/30

8. 付録

No.2231 (LED マトリクス制御) のプログラムを以下に示す。

8.1 Display.h

```
1  #ifndef DISPLAY_H
2  #define DISPLAY_H
3
4  #include <Arduino.h>
5  #include "Arduino_LED_Matrix.h"
6  #include <WiFiS3.h>
7
8  #define BPM_STEP_1    60
9  #define BPM_STEP_2    90
10 #define BPM_STEP_3    120
11 #define BPM_STEP_4    150
12 #define BPM_STEP_5    180
13
14 #define LOCAL_PORT     2390
15 #define HEADER_BPM     'B'
16 #define HEADER_END     'E'
17
18 #define FONT_WIDTH     3
19 #define FONT_HEIGHT    5
20
21 #define MATRIX_ROWS    8
22 #define MATRIX_COLS    12
23
24 void displaySetup();
25 void checkUDP();
26 void displayBPM(int bpm);
27 void clearDisplay();
28 void drawDigit(int digit, int x_offset);
29
30 extern const uint8_t font3x5[10][15];
31 extern ArduinoLEDMatrix matrix;
32 extern WiFiUDP Udp;
33
34 #endif
```

8.2 Display.ino

```
1  #include "Display.h"
2
3  WiFiUDP Udp;
4
5  const char SSID[] = "";
6  const char PASS[] = "";
7
8  IPAddress localIP(192, 168, 11, 10);
9  IPAddress gateway(192, 168, 11, 1);
10 IPAddress subnet(255, 255, 255, 0);
11
12 void setup() {
13     displaySetup();
14
15     if (SSID[0] != '\0') {
16         WiFi.config(localIP, gateway, subnet);
17         while (WiFi.begin(SSID, PASS) != WL_CONNECTED) {
18             delay(1000);
19         }
20     }
21     Udp.begin(LOCAL_PORT);
22 }
23
24 void loop() {
25     checkUDP();
26 }
```

8.3 Display.cpp

```
1  #include "Display.h"
2  #include "Arduino_LED_Matrix.h"
3
4  const uint8_t font3x5[10][15] = {
5      {1,1,1, 1,0,1, 1,0,1, 1,0,1, 1,1,1}, // 0
6      {0,1,0, 1,1,0, 0,1,0, 0,1,0, 1,1,1}, // 1
7      {1,1,1, 0,0,1, 1,1,1, 1,0,0, 1,1,1}, // 2
8      {1,1,1, 0,0,1, 0,1,1, 0,0,1, 1,1,1}, // 3
9      {1,0,1, 1,0,1, 1,1,1, 0,0,1, 0,0,1}, // 4
10     {1,1,1, 1,0,0, 1,1,1, 0,0,1, 1,1,1}, // 5
11     {1,1,1, 1,0,0, 1,1,1, 1,0,1, 1,1,1}, // 6
```

```

12 {1,1,1, 0,0,1, 0,0,1, 0,0,1, 0,0,1}, // 7
13 {1,1,1, 1,0,1, 1,1,1, 1,0,1, 1,1,1}, // 8
14 {1,1,1, 1,0,1, 1,1,1, 0,0,1, 1,1,1} // 9
15 };
16
17 static const int bpmTable[5] = {
18     BPM_STEP_1, BPM_STEP_2, BPM_STEP_3,
19     BPM_STEP_4, BPM_STEP_5
20 };
21
22 ArduinoLEDMatrix matrix;
23
24 void displaySetup() {
25     matrix.begin();
26     uint32_t warmUp[3] = {0, 0, 0};
27     matrix.loadFrame(warmUp);
28     delay(200);
29 }
30
31 static uint8_t frameBuffer[MATRIX_ROWS][MATRIX_COLS];
32 static int currentBPM = -1;
33
34 void checkUDP() {
35     int packetSize = Udp.parsePacket();
36     if (packetSize <= 0) return;
37
38     uint8_t buffer[2] = {0};
39     int len = Udp.read(buffer, sizeof(buffer));
40     if (len < 2) return;
41
42     char header = (char)buffer[0];
43     uint8_t stage = buffer[1];
44
45     if (header == HEADER_END) {
46         clearDisplay();
47         return;
48     }
49
50     if (header != HEADER_BPM) return;
51     if (stage < 1 || stage > 5) return;
52
53     int bpm = bpmTable[stage - 1];
54
55     if (bpm != currentBPM) {
56         currentBPM = bpm;
57         displayBPM(bpm);
58     }
59 }
60
61 void displayBPM(int bpm) {
62     memset(frameBuffer, 0, sizeof(frameBuffer));
63
64     int digits[3];
65     int numDigits;
66
67     if (bpm >= 100) {
68         numDigits = 3;
69         digits[0] = bpm / 100;
70         digits[1] = (bpm / 10) % 10;
71         digits[2] = bpm % 10;
72     } else if (bpm >= 10) {
73         numDigits = 2;
74         digits[0] = bpm / 10;
75         digits[1] = bpm % 10;
76     } else {
77         numDigits = 1;
78         digits[0] = bpm;
79     }
80
81     int totalWidth = numDigits * FONT_WIDTH
82                     + (numDigits - 1) * 1;
83     int startX = (MATRIX_COLS - totalWidth) / 2;
84
85     for (int i = 0; i < numDigits; i++) {
86         drawDigit(digits[i],
87                 startX + i * (FONT_WIDTH + 1));
88     }
89
90     uint32_t bitmap[3] = {0, 0, 0};
91     for (int row = 0; row < MATRIX_ROWS; row++) {
92         for (int col = 0; col < MATRIX_COLS; col++) {
93             if (frameBuffer[row][col]) {
94                 int n = row * MATRIX_COLS + col;
95                 bitmap[n / 32] |=
96                     (1UL << (31 - (n % 32)));
97             }
98         }
99     }

```

```

99     }
100
101     matrix.loadFrame(bitmap);
102 }
103
104 void drawDigit(int digit, int x_offset) {
105     if (digit < 0 || digit > 9) return;
106
107     const int y_offset =
108         (MATRIX_ROWS - FONT_HEIGHT) / 2;
109
110     for (int y = 0; y < FONT_HEIGHT; y++) {
111         for (int x = 0; x < FONT_WIDTH; x++) {
112             int col = x_offset + x;
113             int row = y_offset + y;
114             if (col >= 0 && col < MATRIX_COLS
115                 && row >= 0 && row < MATRIX_ROWS) {
116                 framebuffer[row][col] =
117                     font3x5[digit][y * FONT_WIDTH + x];
118             }
119         }
120     }
121 }
122
123 void clearDisplay() {
124     currentBPM = -1;
125     uint32_t blank[3] = {0, 0, 0};
126     matrix.loadFrame(blank);
127 }

```